

MODULE 3

Training Supervised Deep Learning Networks

3.2 Training Convolution Neural Networks

Training supervised deep neural network is formulated in terms of minimizing a loss function. In this context, training a supervised deep neural network means searching a set of values of parameters (or weights) of the network at which the loss function has minimum value. Gradient descent is an optimization technique which is used to minimize the error by calculating gradients necessary to update the values of the parameters of the network.

The most common and successful learning algorithm for deep learning models is gradient descent-based backpropagation in which error is propagated backward from last layer to the first layer. In this learning technique, all the weights of a neural network are either initialized randomly or initialized by using probability distribution. An input is fed through the network to get the output. The obtained output and the desired output are then used to calculate the error using some cost function (error function). To understand the working of backpropagation, consider a small Convolution Neural Network (CNN) model shown in Fig. 3.1. The CNN model

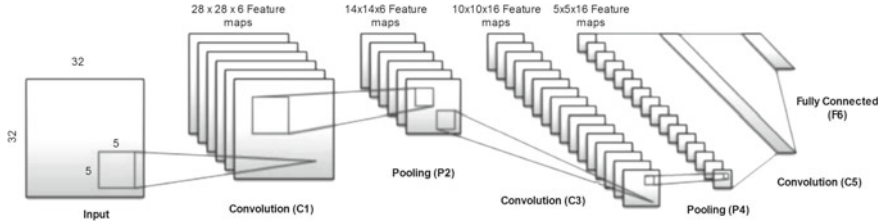


Fig. 3.1 CNN architecture

is similar to LeNet but uses ReLU (which has become a standard in deep networks) as activation function, max-pooling, instead of average pooling. The CNN model consists of three convolution layers, two subsampling layers, and one fully connected layer producing 10 outputs. The first convolution layer convolves input of size 32×32 with six 5×5 filters producing six 28×28 feature maps. Next, pooling layer down-samples these six 28×28 feature maps to size 14×14 . Second convolution layer convolves the down-sampled feature maps with 5×5 filters producing 16 feature maps of size 10×10 . Second pooling layer down-samples the input producing 16 feature maps of size 5×5 . The third convolution layer has the input of size 5×5 and the filter of size 5×5 , so no stride happens. We can say that this third layer has 120 filters of size 5×5 fully connected to each of the 16 feature maps of size 5×5 . Last layer in the architecture is a fully connected layer containing 10 units for 10 classes. The output of the fully connected layer is fed to cost function (Softmax) to calculate the error. Let us describe the CNN architecture in detail including its mathematical operations.

Layer 1 (C1) is a convolution layer which takes input of size 32×32 and convolves it with six filters of size 5×5 producing six feature maps of size 28×28 .

Mathematically, the above operation can be given as

$$C1_{i,j}^k = \left(\sum_{m=0}^4 \sum_{n=0}^4 w_{m,n}^k * I_{i+m,j+n} + b^k \right) \quad (3.1)$$

where $C1^k$ represent six output feature maps of convolution layer C1, k represents filter number, (m, n) are the indices of k th filter and (i, j) are the indices of output.

Equation 3.1 is the mathematical form of convolution operation. The output of convolution operation is fed to an activation function to make the output of linear operation nonlinear. In deep networks, the most successful activation function is ReLU given by

$$\sigma(x) = \max(0, x) \quad (3.2)$$

Using Eq. 3.1, we get

$$C1_{i,j}^k = \sigma \left(\sum_{m=0}^4 \sum_{n=0}^4 w_{m,n}^k * I_{i+m,j+n} + b^k \right) \quad (3.3)$$

Equation 3.3 is the output of layer C1.

Layer 2(P2) is a max-pooling layer. The output of the convolution layer C1 is fed to max-pooling layer. The max-pooling layer takes six feature maps from C1 and performs max-pooling operation on each of them.

The max-pooling operation on $C1^k$ is given as

$$P2^k = \text{MaxPool}(C1^k)$$

For each feature map in $C1^k$, max-pooling performs the following operation:

$$P2_{i,j}^k = \max \left(\begin{matrix} C1_{(2i,2j)}^k, C1_{(2i+1,2j)}^k \\ C1_{(2i,2j+1)}^k, C1_{(2i+1,2j+1)}^k \end{matrix} \right) \quad (3.4)$$

where (i, j) are the indices of k th feature map of output, and k is the feature map index.

Layer 3 (C3) is the second convolution layer which produces 16 feature maps of size 10×10 and is given by

$$C3_{i,j}^k = \sigma \left(\sum_{d=0}^5 \sum_{m=0}^4 \sum_{n=0}^4 w_{m,n}^{k,d} * P2_{i+m,j+n}^d + b^k \right)$$

where $C3^k$ represents the 16 output feature maps of convolution layer C3, k is the index of the output feature map, (m, n) are the indices of filter weights, (i, j) are the indices of output, and d is the index of the number of channels in the input.

Layer 4 (P4) is a max-pooling layer which produces 16 feature maps $P4^k$ of size 5×5 .

The max-pooling operation is given as

$$P4^k = \text{MaxPool}(C3^k)$$

$$P4_{i,j}^k = \max \left(\begin{matrix} C3_{(2i,2j)}^k, C3_{(2i+1,2j)}^k \\ C3_{(2i,2j+1)}^k, C3_{(2i+1,2j+1)}^k \end{matrix} \right)$$

where (i, j) are the indices of k th feature map of output, and k is the feature map index.

Layer 5 (C5) is third convolution layer that produces 120 output feature maps and is given by

$$C5_{i,j}^k = \sigma \left(\sum_{d=0}^{15} \sum_{m=0}^4 \sum_{n=0}^4 w_{m,n}^{k,d} * P4_{i+m,j+n}^d + b^k \right)$$

where $C5^k$ represents the 120 output feature maps of convolution layer C5 of size 1×1 , k is the index of the output feature map, (m, n) are the indices of filter weights, d is the index of the number of channels in input and (i, j) are the indices of output, since output is only 1×1 , the index (i, j) remains $(0, 0)$ for each filter. This formula can be simplified as the filter size is equal to the size of input, so no convolution stride happens

$$C5^k = \sigma \left(\sum_{d=0}^{15} \sum_{m=0}^4 \sum_{n=0}^4 w_{m,n}^{k,d} * P4_{m,n}^d + b^k \right)$$

Layer 6 (F6) is a fully connected layer. It consists of 10 neurons for 10 classes and is mathematically defined as

$$F6^k = \sum_{i=1}^{120} w_i^k * C5^i \quad (3.5)$$

In last layer, for each neuron, the activation function used is softmax function given as

$$Z^k = \text{softmax}(F6^k)$$

The softmax function is defined as

$$Z^k = \text{softmax}(F6^k) = \frac{e^{F6^k}}{\sum_{i=1}^{10} e^{F6^i}} \quad (3.6)$$

The softmax activation function produces the final output of the neurons in the range $[0, 1]$ and all outputs add up to 1. Each of the output represents the probability of the input belonging to a particular class.

Here, Z^k is the vector of size 10 containing final output of the network.

Backward Pass

Loss Layer

During training, an input is fed to the network and output is obtained. The obtained output is compared against the actual output to calculate the error or loss. The calculated error is then used to update the weights of the network. The process is repeated until the error is minimized. The function which is used to calculate the error or loss is called cost/loss function, and there are quite a few loss functions available and for softmax layer the two commonly used loss functions are Mean Squared Error (MSE)

and cross-entropy loss function. The Mean Squared Error (MSE) of the CNN model can be given as

$$\text{loss} = E(Z, \text{target})$$

The loss function E is defined as

$$E(Z, \text{target}) = \frac{1}{10} \sum_{k=1}^{10} (Z^k - \text{target}^k)^2 \quad (3.7)$$

Z^k is the k th output (in this case the network will produce 10 outputs representing class probabilities) generated by the CNN and target^k is the ground truth of the input.

$E(Z, \text{target})$ is the error/loss which represents how far the prediction of the network is from the actual target.

The purpose of training is to minimize the loss function and to do so the weights of the network are continuously updated until the minimum value is achieved. After calculating the error, the gradient of the loss function with respect to each weight of the neural network is calculated. The weights of the network are then updated with their respective gradients. This process is continued until the total error is minimized.

To minimize the loss function E , derivative of E is calculated w.r.t weight w^i . Since the loss function E is defined as composition of functions in series as

$$E = \text{loss}(\text{softmax}(F6(C5(P4(C3(P2(C1(\text{input})))))))$$

That is, the loss function composes of the softmax activation function, which is a function of the fully connected layer $F6$ which in turn is a function of previous layer $C5$ and so on.

The loss function E is differentiable and can be differentiated w.r.t any weight at any layer using chain rule. In backpropagation, the weights of the last layer are updated first followed by second last layer and so on. To start with, let us find the derivative of the cost function w.r.t weight w_i^k at last layer $F6$ using chain rule.

$$\frac{\partial E}{\partial w_i^k} = \frac{\partial E}{\partial Z^k} * \frac{\partial Z^k}{\partial F6^k} * \frac{\partial F6^k}{\partial w_i^k} \quad (3.8)$$

- E is the cost function.
- Z^k represents the output of softmax function.
- $F6^k$ is the output of the last layer.
- k is the output layer neuron index.
- w_i^k is the i th weight of k th neuron of last layer.

After finding the derivative $\frac{\partial E}{\partial w_i^k}$, the weight w_i^k is then updated as

$$w_i^k = w_i^k - \mu \frac{\partial E}{\partial w_i^k} \quad (3.9)$$

where w_i^k is the updated weight and μ is the learning rate.

To solve Eq. 3.8, we first need to find the individual derivatives $\frac{\partial E}{\partial Z^k}$, $\frac{\partial Z^k}{\partial F6^k}$ and $\frac{\partial F6^k}{\partial w_i^k}$, which are given as

$$(i) \quad \frac{\partial E}{\partial Z^k} = \frac{\partial \frac{1}{10} \sum_{k=1}^{10} (Z^k - \text{target}^k)^2}{\partial Z^k} = \frac{1}{5} (Z^k - \text{target}^k)$$

$$(ii) \quad \frac{\partial Z^k}{\partial F6^k} = Z^k * (1 - Z^k)$$

Here, Z^k is the output of softmax function and the derivative of softmax function $f(x)$ is given as $f(x) * (1 - f(x))$ as shown in Sect. 3.3.3.

$$(iii) \quad \frac{\partial F6^k}{\partial w_i^k} = C5^i$$

From Eq. 3.5

$$F6^k = \sum_{i=1}^{120} w_i^k * C5^i = (w_1^k * C5^1 + \dots + w_i^k * C5^i + \dots + w_{120}^k * C5^{120})$$

The derivative of above function is given as

$$\frac{\partial F6^k}{\partial w_i^k} = (0 + \dots + C5^i + \dots + 0) = C5^i$$

Therefore,

$$\frac{\partial E}{\partial w_i^k} = \frac{1}{5} (Z^k - \text{target}^k) * Z^k * (1 - Z^k) * C5^i$$

Here, k is the neuron number in the last layer and i is the index of the weights to that neuron connecting input $C5^i$. For simplification in the backward pass and for propagation of error to previous layers, we calculate the deltas for this layer which is represented by

$$\delta F6^k = \frac{1}{5} (Z^k - \text{target}^k) * Z^k * (1 - Z^k)$$

These delta values enable the calculation of the gradient as well as the process of propagation of this error value to previous layers. The gradient then becomes as

$$\frac{\partial E}{\partial w_i^k} = \delta F6^k * C5^i$$

Here, i is the index of the weight to be updated and k is the neuron/filter number of layer $F6$.

The gradient $\frac{\partial E}{\partial w_i^k}$ calculated above can now be used to update the weight w_i^k using the formula

$$w_i^k = w_i^k - \mu \frac{\partial E}{\partial w_i^k}$$

Hidden Layer (C5)

The backward pass will continue to update the weights in hidden layer $C5$. The equations during forward pass at the layer are given as

$$C5^k = \sigma \left(\sum_{d=0}^{15} \sum_{m=0}^4 \sum_{n=0}^4 w_{m,n}^{k,d} * P4_{m,n}^d + b^k \right)$$

where $C5^k$ represents the 120 feature maps of convolution layer $C5$, (m, n) are the indices of k th filter and d is the channel number.

In order to update the weight $w_{m,n}^{k,d}$, we first have to backpropagate the error from the layer $F6$ to layer $C5$ and then calculate the deltas of layer $C5$. The error backpropagated at this layer $C5$ from layer $F6$ is given by this

$$e^k = \sum_{l=1}^{10} \delta F6^l * w_k^l$$

Here, w_k^l is the weight of the $F6$ layer connecting output of the layer $C5$ to the layer $F6$; k is the index of the 120 neurons in layer $C5$, and l is the index of delta vector at layer $F6$.

The deltas for this layer are given by the formula

$$\delta C5^k = e^k * \text{ReLU}'(x^k)$$

$$\text{ReLU}'(x^k) = \{0 \text{ if } x^k < 0, 1 \text{ if } x^k \text{ otherwise}\}$$

Here, x^k is the summation of the inputs to the neuron multiplied with the weights of this layer; it is the input to the neuron before the activation function during the forward pass, and k is the index of the number of filter at this layer.

The gradient of any weight in this layer $C5$ is given by

$$\frac{\partial E}{\partial w_{m,n}^{k,d}} = \delta C5^k * P4_{m,n}^d$$

Here, d is the channel number and (m, n) are the indices of filter weights; k is the filter number in this layer $C5$.

Hidden Layer (C3)

To calculate the gradient of any of the weights in this layer, we have to backpropagate the error from layer $C5$ through pooling layer $P4$ to layer $C3$.

$$e_{i,j}^k = \sum_{l=1}^{120} \delta C5^l * w_{i,j}^{l,k}$$

Here, $w_{i,j}^{l,k}$ is the weights of the next layer $C5$, l represents the filter number at that layer, k is the channel number/filter number connecting the output of layer $C3$ to $C5$, and (i, j) are the indices of the filter weights at $C5$, as well as the indices of the error matrix at $C3$ layer. Since we propagate error from $C5$ to $C3$ through $P4$ pooling layer, the error value is calculated for only those features which are selected during the max-pooling operation in forward pass.

This is used to calculate the delta values for this layer $C3$. These delta values are then used to calculate the gradient for any weight in this layer $C3$

$$\delta C3_{i,j}^k = e_{i,j}^k * \text{ReLU}'(x_{i,j}^k)$$

Here, $x_{i,j}^k$ is the summation of the summation of the inputs to the neuron multiplied with the weights of this layer; it is the result that is input to the activation function during the forward pass.

The gradient for the weights in this layer is calculated using the formula

$$\frac{\partial E}{\partial w_{m,n}^{k,d}} = \sum_{i=0}^9 \sum_{j=0}^9 \delta C3_{i,j}^k * P2_{m+i,n+j}^d$$

Here, $w_{m,n}^{k,d}$ is the weights of this layer, k is the filter number, d is the channel number, (m, n) are the indices of the weights and (i, j) are the indices of the delta matrix.

Hidden Layer (C1)

To calculate the gradient of any of the weights in this layer, we have to backpropagate the error from layer $C3$ through pooling layer $P2$ to layer $C1$. This operation is implemented by full convolution of the delta matrix with the 180° flipped weight matrix.

$$e^k = \sum_{l=1}^{16} \delta C3^l * w^{l,k}(\text{flipped } 180^\circ)$$

Here, e^k is the error matrix for each of the k filters; k goes from 1 to 6 as there are six filters in layer $C1$. The size of this error matrix is equal to the size of output of this layer, which is 28×28 . The indices (i, j) for each error value of every matrix e^k go from 0 to 27. This is used to calculate the delta values for this layer $C1$. These delta values are then used to calculate the gradient for any weight in this layer $C1$

$$\delta C1_{i,j}^k = e_{i,j}^k * \text{ReLU}'(x_{i,j}^k)$$

Here, $x_{i,j}^k$ is the summation of the inputs to the neuron multiplied with the weights of this layer; it is the result that is input to the activation function during the forward pass, and (i, j) is the index of the delta matrix which goes from 0 to 27.

The gradient for the weights in this layer is calculated using the formula

$$\frac{\partial E}{\partial w_{m,n}^k} = \sum_{i=0}^{27} \sum_{j=0}^{27} \delta C 1_{i,j}^k * \text{input image}_{m+i,n+j}$$

Here, $w_{m,n}^k$ is the weights of this layer, k is the filter number, (m, n) are the indices of the weights and (i, j) are the indices of the delta matrix.

3.4 Gradient Descent-Based Optimization Techniques

Gradient descent is an optimization technique used to minimize/maximize the cost function by calculating gradients necessary to update the values of the parameters of the network. Various variants of this optimizing technique define how to calculate the parameter updates using these gradients.

3.4.1 Gradient Descent Variants

There are three commonly used Gradient Descent (GD) variants. These variants differ in how many training examples are used to compute the gradient. The three variants are explained as follows.

3.4.1.1 Batch Gradient Descent (GD)

In traditional Gradient Descent (GD), also known as batch gradient descent, error gradient with respect to weight parameter w is computed for the entire training set followed by updating the weight parameter as shown below:

$$w = w - \mu \cdot \nabla E(w)$$

where $\nabla E(w)$ is the error gradient with respect to weight w and μ is the learning rate that defines the step size to take along the gradient. The learning rate is a hyperparameter which cannot be too high or too low. Large value of learning rate can miss the optimum value, and too low learning rate will result in slow training time.

The training set often contains hundreds and thousands of examples that may demand huge memory which makes it difficult to fit in the memory. As a result, computing the error gradient can be very slow.

3.4.1.2 Stochastic Gradient Descent (SGD)

The above problem can be rectified by using Stochastic Gradient Descent (SGD), also known as incremental gradient descent, where gradient is computed for one training example at a time followed by updating of parameter values. It is usually much faster than standard gradient descent as it performs one update at a time.

$$w = w - \mu \cdot \nabla E(w; \mathbf{x}(\mathbf{i}); \mathbf{y}(\mathbf{i}))$$

where $\nabla E(w; \mathbf{x}(\mathbf{i}); \mathbf{y}(\mathbf{i}))$ is the gradient of loss function— $E(w)$ w.r.t parameters w , for the training example $\{x(i), y(i)\}$.

In SGD, the one example based on updation of the parameter values causes the loss function to fluctuate frequently.

3.4.1.3 Mini-batch Gradient Descent

Mini-batch gradient descent also known as mini-batch SGD is a combination of both standard gradient descent and SGD techniques. Mini-batch SGD divides the entire training set into mini-batches of n training examples and performs the updating of parameter values for each mini-batch. This type of gradient descent technique takes advantage of both standard gradient descent and SGD techniques, and is commonly used optimization technique in deep learning.

$$w = w - \mu \cdot \nabla E(w; \mathbf{x}(\mathbf{i} : \mathbf{i} + \mathbf{n}); \mathbf{y}(\mathbf{i} : \mathbf{i} + \mathbf{n}))$$

Typical mini-batch size varies from 50 to 256 and should also be chosen sensibly according to the following factors:

- Large batch sizes provide more accurate gradients but have high memory requirements.
- Small batch sizes can offer a regularizing effect but require a small learning rate to maintain stability owing to the high variance in the estimate of the gradient. This in turn increases the training time because of the reduced learning rate.

3.4.2 Improving Gradient Descent for Faster Convergence

The main objective of optimization is to minimize the cost/loss or objective function. There are many methods available that help an optimization algorithm to converge faster. Some of the commonly used methods are discussed below.

3.4.2.1 AdaGrad

In SGD, the learning rate is set independently of gradients which may sometimes cause problems. For example, if the gradient is large, large learning rate would result in large step size, which means it may not achieve the optimum value as it may keep oscillating around the optimum value, and if the magnitude of gradient is small, a small learning rate may result in slow convergence. The problem can be resolved by using some adaptive approach for setting the learning rate. AdaGrad is one such adaptive model which uses adaptive learning rate by adding squared norms of previous gradients and dividing the learning rate by the square root of this sum.

$$w_{t+1,i} = w_{t,i} - \frac{\mu}{\sqrt{G_i}} \cdot \nabla_{t,i}$$

where $\nabla_{t,i}$ is the gradient of loss function with respect to parameter w_i and $G_i = \sum_{\tau=1}^t \nabla_{\tau}^2$ and ∇_{τ} is gradient at iteration τ .

In this way, parameters with high gradients will have small effective learning rate and parameters with small gradients receive increased effective learning rate.

The main advantage of AdaGrad is that learning rate is automatically adjusted, and there is no need to manually tune it. However, the sum in the denominator keeps on increasing which gradually causes the learning rate to decay. This decaying learning rate can slow down the learning or stop the learning completely.

3.4.2.2 AdaDelta

AdaDelta is a modified version of AdaGrad which overcomes the problem of decaying learning rate. AdaDelta limits the number of previous gradients to some fixed size x and then the average of these past gradients is stored for efficiency. The average value $\text{Avg}(\nabla_t^2)$ at time t only depends on previous average and current gradient. The parameter update is then made as

$$w_{t+1} = w_t - \frac{\mu}{\sqrt{\text{Avg}(\nabla_t^2)}} \cdot \nabla_t$$

Since the denominator is just the Root Mean Square (RMS) of the parameters

$$w_{t+1} = w_t - \frac{\mu}{\text{RMS}(\nabla_t)} \cdot \nabla_t$$

3.4.2.3 RMSProp

The vanishing learning rate in AdaGrad can be rectified by using RMSProp. It is a modified version of AdaGrad which discards history from the distant past by introducing exponentially weighted moving average. RMSProp uses sign of the gradient instead of the magnitude of the gradient to update the weights. The working of RMSProp optimizer is as follows:

- (a) Set same magnitude of updates for all weights. Set maximum and minimum allowable weight updates to $\Delta_{\max}$ and $\Delta_{\min}$, respectively.
- (b) At each iteration, if signs of current gradient and previous gradient are same, then increase learning rate by a factor of 1.2, i.e. $\eta = \eta + 1.2$.
Therefore, the update Δ_{ij}^{t+1} becomes

$$\Delta_{ij}^{t+1} = \min(\eta + \Delta_{ij}^t, \Delta_{\max})$$

- (c) If signs of current gradient and previous gradient are different, then decrease the learning rate by a factor of 0.5, i.e., $\eta = \eta - 0.5$

$$\Delta_{ij}^{t+1} = \max(\eta - \Delta_{ij}^t, \Delta_{\min})$$

3.4.2.4 Adam

Adaptive Moment Estimation (Adam) is an adaptive optimization technique that takes advantages of AdaGrad and RMSProp. Like AdaDelta and RMSProp, Adam saves an exponentially decaying average of previous squared gradients v_t . In addition to that, Adam also computes the average of the second moments of the gradients m_t .

m_t and v_t which are values of the mean and uncentered variance, respectively, are given as

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Adam updates exponential moving averages of the gradient and the squared gradient where the hyperparameters $\beta_1, \beta_2 \in [0, 1]$ control the decay rates of these moving averages.

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

The final formula for update is given as

$$w_{t+1} = w_t - \frac{\mu}{\sqrt{\hat{v}_t}} \cdot \hat{m}_t$$

Adam slightly performs better than other adaptive techniques and converges very fast. It also overcomes the problems faced by other optimization techniques such as decaying learning rate, high variance in updates, and slow convergence.

Note that a smoothing term 'e' is usually added in the denominator of the above fast converging methods to avoid divide by 0.

3.5 Challenges in Training Deep Networks

Training a deep neural network is a challenging task, and some of the prominent challenges in training deep models are discussed below.

3.5.1 Vanishing Gradient

Any deep neural network with activation function like sigmoid, $\tanh$, etc. and training through backpropagation suffers from *vanishing gradient* problem. Vanishing gradient makes it very hard to train and update the parameters of the initial layers in the network. This problem worsens as the number of layers in the network increases. The aim of backpropagation in neural networks is to update the parameters such that the error of the network is minimized and actual output gets closer to the target output. During backpropagation, the weights are updated using gradient descent (rate

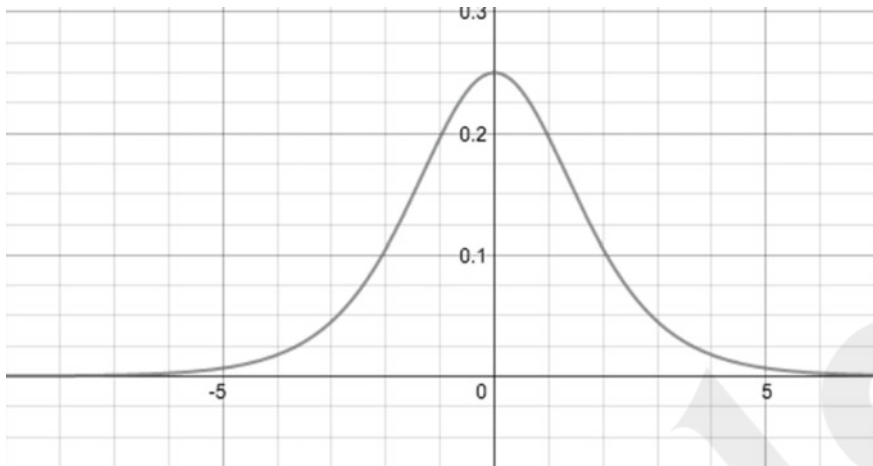


Fig. 3.2 Graph of derivative of sigmoid function

of change in total error E with respect to any weight w). In deep networks, these gradients determine how much each weight should change. The gradients become smaller as they propagate through many layers. The sigmoid function is given by

$$f(x) = \frac{1}{1 + e^{-x}}$$

The derivative of this sigmoid function is given as

$$f'(x) = \frac{1}{1 + e^{-x}} \left(1 - \frac{1}{1 + e^{-x}} \right)$$

The graph of the above equation is given in Fig. 3.2. It is evident from the graph that the maximum point of the function is 0.25 implying that the output of the derivative of the cost function will always lie between 0 and 0.25. In other words, the errors will be squeezed to the range 0 and 0.25 at each layer. Therefore, the gradients become smaller and smaller after each layer and finally vanish leaving top layers untrained.

The vanishing gradient is the primary reason that makes sigmoid or tanh activations unsuitable for deep networks, and this is where Rectified Linear Units (ReLUs) come to the rescue. ReLU activation function does not suffer from vanishing gradient because there is no squeezing of inputs as the derivative is always 1 for positive inputs. A Rectified Linear Unit (ReLU) outputs 0 for input less than 0 and raw output otherwise. That is, if the input x is less than 0, then the output is 0 and if x is greater than 0, the output is equal to the input x and its derivative is 1.

That is $f(x) = x$ and $f'(x) = 1$ for $x > 0$.

3.5.2 Training Data Size

Deep networks use training data for learning and are capable of learning complex nonlinear relationships between input data and output label. Deep networks require a large number of parameters to be learnt before it can be utilized to deliver the desired result. The number of parameters in deep models is large. More complex models mean more powerful abstraction, and more parameters which require more data. So, the size of training data is an important factor which can influence the success of deep models. In fact, all the successful deep models have been trained on some very large dataset. For example, AlexNet, GoogleNet, VGG, ResNet, etc. all have been trained on a vast dataset of images called ImageNet. ImageNet is an image dataset which contains around 1.2 million labeled images distributed over 1000 classes. However, one can argue that deep models for object recognition and detection require large number of parameters to deal with different variations, different poses, different variations in color, etc., and thus require vast size dataset for training. On the other hand, less complex problems (like classification of medical images) where the variations are very small as compared to variations mentioned above can be solved using less complex models which do not require huge training datasets. The claim is true to some extent, but model complexity alone cannot decide the size of the data required for training. Training data quality also plays an important role in it. Noisy data means low Signal-to-Noise Ratio (SNR) in the data and lower SNR means more data is required for convergence. Therefore, the size of dataset really depends on the complexity of the problem being studied, and the quality of data.

Irrespective of the complexity of the task, large training data size can significantly improve the performance of deep models. That is, the larger the training data size, the better the accuracy. But the question “How much data is enough?” still remains unanswered, and there is no rule of thumb that can define exact number of examples required to train a particular deep model.

3.5.3 Overfitting and Underfitting

Once a model is trained on a training dataset, it is expected to perform well on new, previously unseen data which was not present during learning. The ability of a machine learning model to perform well on new and unseen data is called **generalization**. Generalization is one of the objectives of a good deep learning model. To estimate the generalization ability of a deep learning model, it is tested on data collected separately from the training set.

Deep learning models can suffer from two problems, viz. overfitting and underfitting and both can lead to poor model performance.

Overfitting occurs when a model is trained, and it performs so well on training data that it is unable to generalize to new data. That is, the model has low training

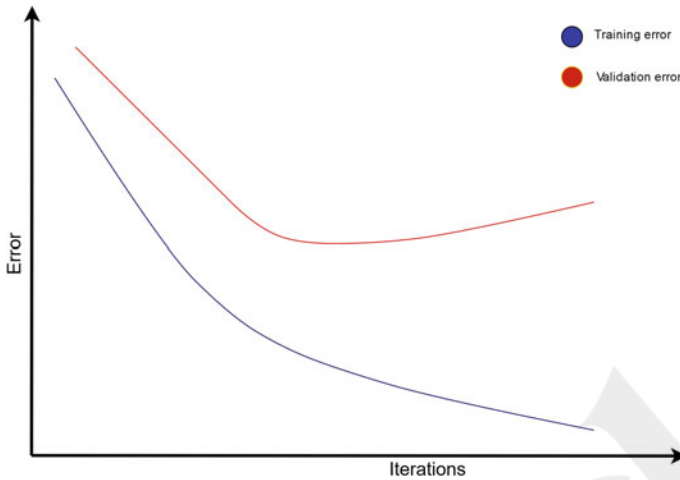


Fig. 3.3 Overfitting: training error (blue), validation error (red) as a function of the number of iterations

error but is unable to achieve low test error. In this case, the model is memorizing the data instead of learning. See Fig. 3.3.

Underfitting occurs when a model is not able to learn properly and its performance on the training set is poor.

The most common problem in deep learning is overfitting. Deep models like Convolutional Neural Network (ConvNet) have a large number of learnable parameters that must be learnt before they can be utilized to perform the task of interest. In order to train these models, extensively large training data with a specific end goal is required to accomplish the desired performance. If the training data is too small compared to the number of weights to be learned, then the network suffers from overfitting.

Overfitting is a common problem in deep networks, however, there are few techniques available that can be used in deep learning models to limit overfitting:

- (a) Increase the training dataset.
- (b) Reducing network size.
- (c) Data augmentation: Modifying the current training data in a random way (Scaling, zooming, translation, etc.) to generate more training data.
- (d) Interpolate weight penalties like $L1$ and $L2$ regularization and soft weight sharing.
- (e) Dropout: The most popular technique to reduce overfitting is *Dropout*. Dropout refers to dropping out neurons/units in a neural network during training. Dropping a unit means temporarily detaching it from the network including all its

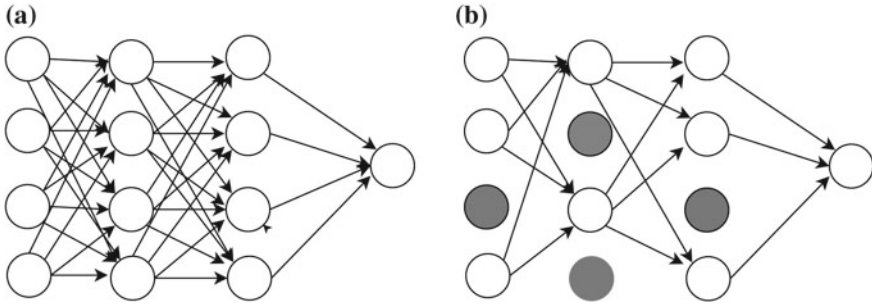


Fig. 3.4 **a** A simple neural network, **b** neural network after dropout

inward and outward connections (Fig. 3.4). The dropped-out neurons neither contribute to the forward pass nor do they contribute in backward pass. By using dropout, the network is forced to learn more robust features as network architecture changes with each input.

3.5.4 High-Performance Hardware

Training deep models on huge datasets require machines with sufficient processing power and memory. In order to get high efficiency and quick training time, it is highly recommended to use multi-core high-performance Graphics Processing Units (GPUs). These high-performance machines, GPUs, and memory are very costly and consume a lot of energy. Therefore, adopting deep learning solutions to real world becomes expensive and energy-consuming task.

4.2 Poor Conditioning

Conditioning refers to how rapidly a function changes with respect to small changes in its inputs. Functions that change rapidly when their inputs are perturbed slightly can be problematic for scientific computation because rounding errors in the inputs can result in large changes in the output.

Consider the function $f(\mathbf{x}) = \mathbf{A}^{-1}\mathbf{x}$. When $\mathbf{A} \in \mathbb{R}^{n \times n}$ has an eigenvalue decomposition, its **condition number** is

$$\max_{i,j} \left| \frac{\lambda_i}{\lambda_j} \right|. \quad (4.2)$$

This is the ratio of the magnitude of the largest and smallest eigenvalue. When this number is large, matrix inversion is particularly sensitive to error in the input.

This sensitivity is an intrinsic property of the matrix itself, not the result of rounding error during matrix inversion. Poorly conditioned matrices amplify pre-existing errors when we multiply by the true matrix inverse. In practice, the error will be compounded further by numerical errors in the inversion process itself.

4.3 Gradient-Based Optimization

Most deep learning algorithms involve optimization of some sort. Optimization refers to the task of either minimizing or maximizing some function $f(\mathbf{x})$ by altering $\mathbf{x}$. We usually phrase most optimization problems in terms of minimizing $f(\mathbf{x})$. Maximization may be accomplished via a minimization algorithm by minimizing $-f(\mathbf{x})$.

The function we want to minimize or maximize is called the **objective function** or **criterion**. When we are minimizing it, we may also call it the **cost function**, **loss function**, or **error function**. In this book, we use these terms interchangeably, though some machine learning publications assign special meaning to some of these terms.

We often denote the value that minimizes or maximizes a function with a superscript $*$. For example, we might say $\mathbf{x}^* = \arg \min f(\mathbf{x})$.

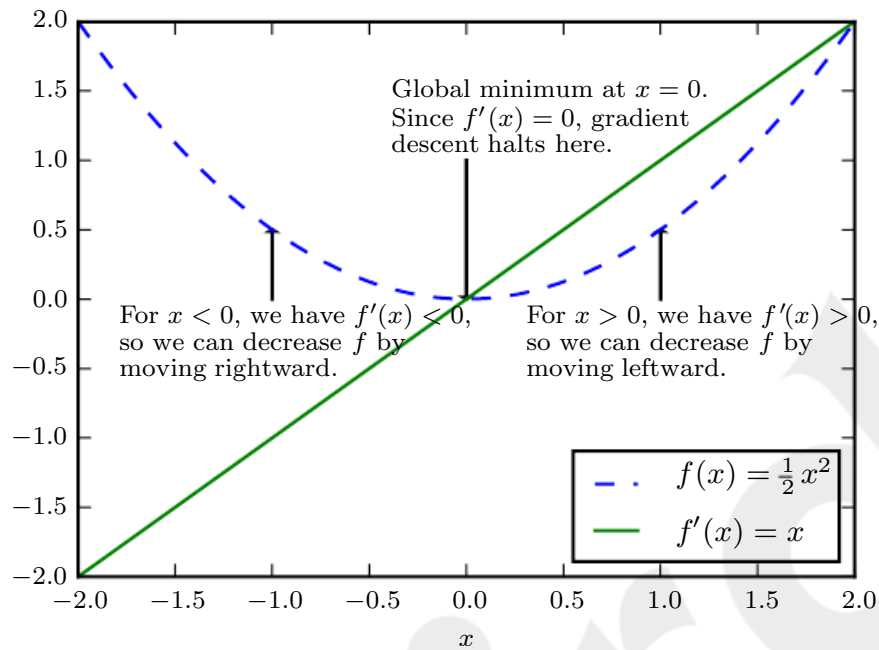


Figure 4.1: An illustration of how the gradient descent algorithm uses the derivatives of a function can be used to follow the function downhill to a minimum.

We assume the reader is already familiar with calculus, but provide a brief review of how calculus concepts relate to optimization here.

Suppose we have a function $y = f(x)$, where both x and y are real numbers. The **derivative** of this function is denoted as $f'(x)$ or as $\frac{dy}{dx}$. The derivative $f'(x)$ gives the slope of $f(x)$ at the point x . In other words, it specifies how to scale a small change in the input in order to obtain the corresponding change in the output: $f(x + \epsilon) \approx f(x) + \epsilon f'(x)$.

The derivative is therefore useful for minimizing a function because it tells us how to change x in order to make a small improvement in y . For example, we know that $f(x - \epsilon \text{sign}(f'(x)))$ is less than $f(x)$ for small enough ϵ . We can thus reduce $f(x)$ by moving x in small steps with opposite sign of the derivative. This technique is called **gradient descent** (Cauchy, 1847). See figure 4.1 for an example of this technique.

When $f'(x) = 0$, the derivative provides no information about which direction to move. Points where $f'(x) = 0$ are known as **critical points** or **stationary points**. A **local minimum** is a point where $f(x)$ is lower than at all neighboring points, so it is no longer possible to decrease $f(x)$ by making infinitesimal steps. A **local maximum** is a point where $f(x)$ is higher than at all neighboring points,

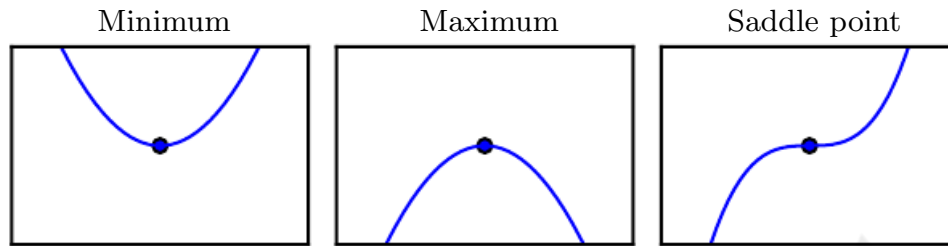


Figure 4.2: Examples of each of the three types of critical points in 1-D. A critical point is a point with zero slope. Such a point can either be a local minimum, which is lower than the neighboring points, a local maximum, which is higher than the neighboring points, or a saddle point, which has neighbors that are both higher and lower than the point itself.

so it is not possible to increase $f(x)$ by making infinitesimal steps. Some critical points are neither maxima nor minima. These are known as **saddle points**. See figure 4.2 for examples of each type of critical point.

A point that obtains the absolute lowest value of $f(x)$ is a **global minimum**. It is possible for there to be only one global minimum or multiple global minima of the function. It is also possible for there to be local minima that are not globally optimal. In the context of deep learning, we optimize functions that may have many local minima that are not optimal, and many saddle points surrounded by very flat regions. All of this makes optimization very difficult, especially when the input to the function is multidimensional. We therefore usually settle for finding a value of f that is very low, but not necessarily minimal in any formal sense. See figure 4.3 for an example.

We often minimize functions that have multiple inputs: $f : \mathbb{R}^n \rightarrow \mathbb{R}$. For the concept of “minimization” to make sense, there must still be only one (scalar) output.

For functions with multiple inputs, we must make use of the concept of **partial derivatives**. The partial derivative $\frac{\partial}{\partial x_i} f(\mathbf{x})$ measures how f changes as only the variable x_i increases at point $\mathbf{x}$. The **gradient** generalizes the notion of derivative to the case where the derivative is with respect to a vector: the gradient of f is the vector containing all of the partial derivatives, denoted $\nabla_{\mathbf{x}} f(\mathbf{x})$. Element i of the gradient is the partial derivative of f with respect to x_i . In multiple dimensions,

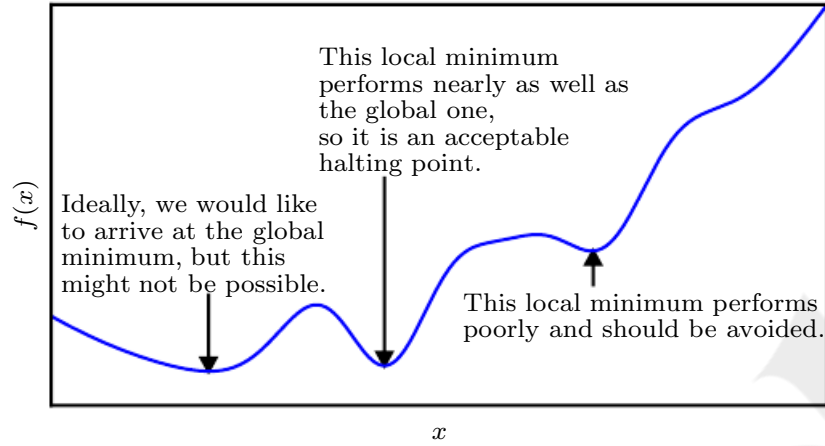


Figure 4.3: Optimization algorithms may fail to find a global minimum when there are multiple local minima or plateaus present. In the context of deep learning, we generally accept such solutions even though they are not truly minimal, so long as they correspond to significantly low values of the cost function.

critical points are points where every element of the gradient is equal to zero.

The **directional derivative** in direction $\mathbf{u}$ (a unit vector) is the slope of the function f in direction $\mathbf{u}$. In other words, the directional derivative is the derivative of the function $f(\mathbf{x} + \alpha\mathbf{u})$ with respect to α , evaluated at $\alpha = 0$. Using the chain rule, we can see that $\frac{\partial}{\partial \alpha} f(\mathbf{x} + \alpha\mathbf{u})$ evaluates to $\mathbf{u}^\top \nabla_{\mathbf{x}} f(\mathbf{x})$ when $\alpha = 0$.

To minimize f , we would like to find the direction in which f decreases the fastest. We can do this using the directional derivative:

$$\min_{\mathbf{u}, \mathbf{u}^\top \mathbf{u} = 1} \mathbf{u}^\top \nabla_{\mathbf{x}} f(\mathbf{x}) \quad (4.3)$$

$$= \min_{\mathbf{u}, \mathbf{u}^\top \mathbf{u} = 1} \|\mathbf{u}\|_2 \|\nabla_{\mathbf{x}} f(\mathbf{x})\|_2 \cos \theta \quad (4.4)$$

where θ is the angle between $\mathbf{u}$ and the gradient. Substituting in $\|\mathbf{u}\|_2 = 1$ and ignoring factors that do not depend on $\mathbf{u}$, this simplifies to $\min_{\mathbf{u}} \cos \theta$. This is minimized when $\mathbf{u}$ points in the opposite direction as the gradient. In other words, the gradient points directly uphill, and the negative gradient points directly downhill. We can decrease f by moving in the direction of the negative gradient. This is known as the **method of steepest descent** or **gradient descent**.

Steepest descent proposes a new point

$$\mathbf{x}' = \mathbf{x} - \epsilon \nabla_{\mathbf{x}} f(\mathbf{x}) \quad (4.5)$$

where ϵ is the **learning rate**, a positive scalar determining the size of the step. We can choose ϵ in several different ways. A popular approach is to set ϵ to a small constant. Sometimes, we can solve for the step size that makes the directional derivative vanish. Another approach is to evaluate $f(\mathbf{x} - \epsilon \nabla_{\mathbf{x}} f(\mathbf{x}))$ for several values of ϵ and choose the one that results in the smallest objective function value. This last strategy is called a **line search**.

Steepest descent converges when every element of the gradient is zero (or, in practice, very close to zero). In some cases, we may be able to avoid running this iterative algorithm, and just jump directly to the critical point by solving the equation $\nabla_{\mathbf{x}} f(\mathbf{x}) = 0$ for $\mathbf{x}$.

Although gradient descent is limited to optimization in continuous spaces, the general concept of repeatedly making a small move (that is approximately the best small move) towards better configurations can be generalized to discrete spaces. Ascending an objective function of discrete parameters is called **hill climbing** (Russel and Norvig, 2003).

4.3.1 Beyond the Gradient: Jacobian and Hessian Matrices

Sometimes we need to find all of the partial derivatives of a function whose input and output are both vectors. The matrix containing all such partial derivatives is known as a **Jacobian matrix**. Specifically, if we have a function $\mathbf{f} : \mathbb{R}^m \rightarrow \mathbb{R}^n$, then the Jacobian matrix $\mathbf{J} \in \mathbb{R}^{n \times m}$ of $\mathbf{f}$ is defined such that $J_{i,j} = \frac{\partial}{\partial x_j} f(\mathbf{x})_i$.

We are also sometimes interested in a derivative of a derivative. This is known as a **second derivative**. For example, for a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the derivative with respect to x_i of the derivative of f with respect to x_j is denoted as $\frac{\partial^2}{\partial x_i \partial x_j} f$.

In a single dimension, we can denote $\frac{d^2}{dx^2} f$ by $f''(x)$. The second derivative tells us how the first derivative will change as we vary the input. This is important because it tells us whether a gradient step will cause as much of an improvement as we would expect based on the gradient alone. We can think of the second derivative as measuring **curvature**. Suppose we have a quadratic function (many functions that arise in practice are not quadratic but can be approximated well as quadratic, at least locally). If such a function has a second derivative of zero, then there is no curvature. It is a perfectly flat line, and its value can be predicted using only the gradient. If the gradient is 1, then we can make a step of size ϵ along the negative gradient, and the cost function will decrease by ϵ . If the second derivative is negative, the function curves downward, so the cost function will actually decrease by more than ϵ . Finally, if the second derivative is positive, the function curves upward, so the cost function can decrease by less than ϵ . See

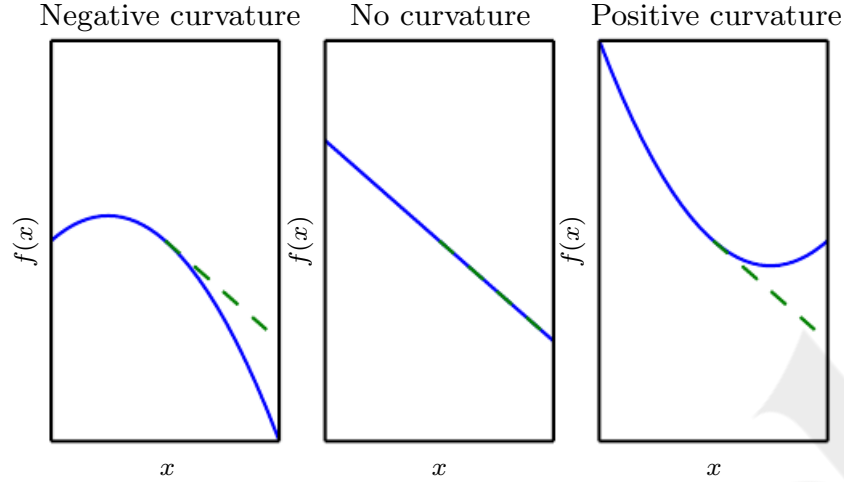


Figure 4.4: The second derivative determines the curvature of a function. Here we show quadratic functions with various curvature. The dashed line indicates the value of the cost function we would expect based on the gradient information alone as we make a gradient step downhill. In the case of negative curvature, the cost function actually decreases faster than the gradient predicts. In the case of no curvature, the gradient predicts the decrease correctly. In the case of positive curvature, the function decreases slower than expected and eventually begins to increase, so steps that are too large can actually increase the function inadvertently.

figure 4.4 to see how different forms of curvature affect the relationship between the value of the cost function predicted by the gradient and the true value.

When our function has multiple input dimensions, there are many second derivatives. These derivatives can be collected together into a matrix called the **Hessian matrix**. The Hessian matrix $\mathbf{H}(f)(\mathbf{x})$ is defined such that

$$\mathbf{H}(f)(\mathbf{x})_{i,j} = \frac{\partial^2}{\partial x_i \partial x_j} f(\mathbf{x}). \quad (4.6)$$

Equivalently, the Hessian is the Jacobian of the gradient.

Anywhere that the second partial derivatives are continuous, the differential operators are commutative, i.e. their order can be swapped:

$$\frac{\partial^2}{\partial x_i \partial x_j} f(\mathbf{x}) = \frac{\partial^2}{\partial x_j \partial x_i} f(\mathbf{x}). \quad (4.7)$$

This implies that $H_{i,j} = H_{j,i}$, so the Hessian matrix is symmetric at such points. Most of the functions we encounter in the context of deep learning have a symmetric Hessian almost everywhere. Because the Hessian matrix is real and symmetric, we can decompose it into a set of real eigenvalues and an orthogonal basis of

eigenvectors. The second derivative in a specific direction represented by a unit vector $\mathbf{d}$ is given by $\mathbf{d}^\top \mathbf{H} \mathbf{d}$. When $\mathbf{d}$ is an eigenvector of $\mathbf{H}$, the second derivative in that direction is given by the corresponding eigenvalue. For other directions of $\mathbf{d}$, the directional second derivative is a weighted average of all of the eigenvalues, with weights between 0 and 1, and eigenvectors that have smaller angle with $\mathbf{d}$ receiving more weight. The maximum eigenvalue determines the maximum second derivative and the minimum eigenvalue determines the minimum second derivative.

The (directional) second derivative tells us how well we can expect a gradient descent step to perform. We can make a second-order Taylor series approximation to the function $f(\mathbf{x})$ around the current point $\mathbf{x}^{(0)}$:

$$f(\mathbf{x}) \approx f(\mathbf{x}^{(0)}) + (\mathbf{x} - \mathbf{x}^{(0)})^\top \mathbf{g} + \frac{1}{2}(\mathbf{x} - \mathbf{x}^{(0)})^\top \mathbf{H}(\mathbf{x} - \mathbf{x}^{(0)}). \quad (4.8)$$

where $\mathbf{g}$ is the gradient and $\mathbf{H}$ is the Hessian at $\mathbf{x}^{(0)}$. If we use a learning rate of ϵ , then the new point $\mathbf{x}$ will be given by $\mathbf{x}^{(0)} - \epsilon \mathbf{g}$. Substituting this into our approximation, we obtain

$$f(\mathbf{x}^{(0)} - \epsilon \mathbf{g}) \approx f(\mathbf{x}^{(0)}) - \epsilon \mathbf{g}^\top \mathbf{g} + \frac{1}{2} \epsilon^2 \mathbf{g}^\top \mathbf{H} \mathbf{g}. \quad (4.9)$$

There are three terms here: the original value of the function, the expected improvement due to the slope of the function, and the correction we must apply to account for the curvature of the function. When this last term is too large, the gradient descent step can actually move uphill. When $\mathbf{g}^\top \mathbf{H} \mathbf{g}$ is zero or negative, the Taylor series approximation predicts that increasing ϵ forever will decrease f forever. In practice, the Taylor series is unlikely to remain accurate for large ϵ , so one must resort to more heuristic choices of ϵ in this case. When $\mathbf{g}^\top \mathbf{H} \mathbf{g}$ is positive, solving for the optimal step size that decreases the Taylor series approximation of the function the most yields

$$\epsilon^* = \frac{\mathbf{g}^\top \mathbf{g}}{\mathbf{g}^\top \mathbf{H} \mathbf{g}}. \quad (4.10)$$

In the worst case, when $\mathbf{g}$ aligns with the eigenvector of $\mathbf{H}$ corresponding to the maximal eigenvalue $\lambda_{\max}$, then this optimal step size is given by $\frac{1}{\lambda_{\max}}$. To the extent that the function we minimize can be approximated well by a quadratic function, the eigenvalues of the Hessian thus determine the scale of the learning rate.

The second derivative can be used to determine whether a critical point is a local maximum, a local minimum, or saddle point. Recall that on a critical point, $f'(x) = 0$. When the second derivative $f''(x) > 0$, the first derivative $f'(x)$ increases as we move to the right and decreases as we move to the left. This means

$f'(x - \epsilon) < 0$ and $f'(x + \epsilon) > 0$ for small enough ϵ . In other words, as we move right, the slope begins to point uphill to the right, and as we move left, the slope begins to point uphill to the left. Thus, when $f'(x) = 0$ and $f''(x) > 0$, we can conclude that x is a local minimum. Similarly, when $f'(x) = 0$ and $f''(x) < 0$, we can conclude that x is a local maximum. This is known as the **second derivative test**. Unfortunately, when $f''(x) = 0$, the test is inconclusive. In this case x may be a saddle point, or a part of a flat region.

In multiple dimensions, we need to examine all of the second derivatives of the function. Using the eigendecomposition of the Hessian matrix, we can generalize the second derivative test to multiple dimensions. At a critical point, where $\nabla_{\mathbf{x}} f(\mathbf{x}) = 0$, we can examine the eigenvalues of the Hessian to determine whether the critical point is a local maximum, local minimum, or saddle point. When the Hessian is positive definite (all its eigenvalues are positive), the point is a local minimum. This can be seen by observing that the directional second derivative in any direction must be positive, and making reference to the univariate second derivative test. Likewise, when the Hessian is negative definite (all its eigenvalues are negative), the point is a local maximum. In multiple dimensions, it is actually possible to find positive evidence of saddle points in some cases. When at least one eigenvalue is positive and at least one eigenvalue is negative, we know that $\mathbf{x}$ is a local maximum on one cross section of f but a local minimum on another cross section. See figure 4.5 for an example. Finally, the multidimensional second derivative test can be inconclusive, just like the univariate version. The test is inconclusive whenever all of the non-zero eigenvalues have the same sign, but at least one eigenvalue is zero. This is because the univariate second derivative test is inconclusive in the cross section corresponding to the zero eigenvalue.

In multiple dimensions, there is a different second derivative for each direction at a single point. The condition number of the Hessian at this point measures how much the second derivatives differ from each other. When the Hessian has a poor condition number, gradient descent performs poorly. This is because in one direction, the derivative increases rapidly, while in another direction, it increases slowly. Gradient descent is unaware of this change in the derivative so it does not know that it needs to explore preferentially in the direction where the derivative remains negative for longer. It also makes it difficult to choose a good step size. The step size must be small enough to avoid overshooting the minimum and going uphill in directions with strong positive curvature. This usually means that the step size is too small to make significant progress in other directions with less curvature. See figure 4.6 for an example.

This issue can be resolved by using information from the Hessian matrix to guide

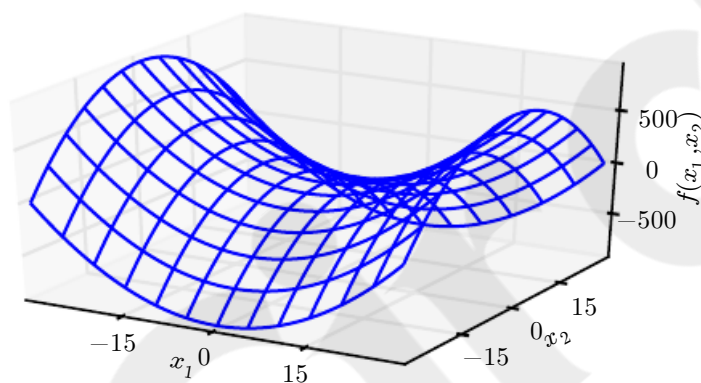


Figure 4.5: A saddle point containing both positive and negative curvature. The function in this example is $f(\mathbf{x}) = x_1^2 - x_2^2$. Along the axis corresponding to x_1 , the function curves upward. This axis is an eigenvector of the Hessian and has a positive eigenvalue. Along the axis corresponding to x_2 , the function curves downward. This direction is an eigenvector of the Hessian with negative eigenvalue. The name “saddle point” derives from the saddle-like shape of this function. This is the quintessential example of a function with a saddle point. In more than one dimension, it is not necessary to have an eigenvalue of 0 in order to get a saddle point: it is only necessary to have both positive and negative eigenvalues. We can think of a saddle point with both signs of eigenvalues as being a local maximum within one cross section and a local minimum within another cross section.

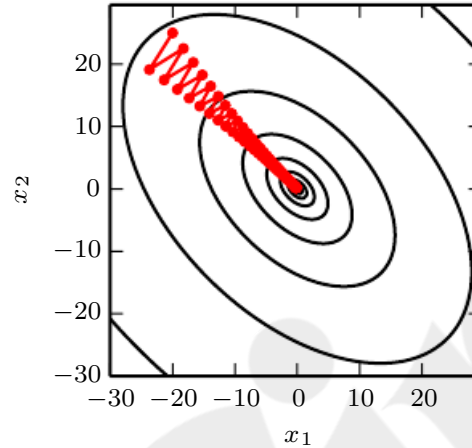


Figure 4.6: Gradient descent fails to exploit the curvature information contained in the Hessian matrix. Here we use gradient descent to minimize a quadratic function $f(\mathbf{x})$ whose Hessian matrix has condition number 5. This means that the direction of most curvature has five times more curvature than the direction of least curvature. In this case, the most curvature is in the direction $[1, 1]^\top$ and the least curvature is in the direction $[1, -1]^\top$. The red lines indicate the path followed by gradient descent. This very elongated quadratic function resembles a long canyon. Gradient descent wastes time repeatedly descending canyon walls, because they are the steepest feature. Because the step size is somewhat too large, it has a tendency to overshoot the bottom of the function and thus needs to descend the opposite canyon wall on the next iteration. The large positive eigenvalue of the Hessian corresponding to the eigenvector pointed in this direction indicates that this directional derivative is rapidly increasing, so an optimization algorithm based on the Hessian could predict that the steepest direction is not actually a promising search direction in this context.

the search. The simplest method for doing so is known as **Newton's method**. Newton's method is based on using a second-order Taylor series expansion to approximate $f(\mathbf{x})$ near some point $\mathbf{x}^{(0)}$:

$$f(\mathbf{x}) \approx f(\mathbf{x}^{(0)}) + (\mathbf{x} - \mathbf{x}^{(0)})^\top \nabla_{\mathbf{x}} f(\mathbf{x}^{(0)}) + \frac{1}{2} (\mathbf{x} - \mathbf{x}^{(0)})^\top \mathbf{H}(f)(\mathbf{x}^{(0)}) (\mathbf{x} - \mathbf{x}^{(0)}). \quad (4.11)$$

If we then solve for the critical point of this function, we obtain:

$$\mathbf{x}^* = \mathbf{x}^{(0)} - \mathbf{H}(f)(\mathbf{x}^{(0)})^{-1} \nabla_{\mathbf{x}} f(\mathbf{x}^{(0)}). \quad (4.12)$$

When f is a positive definite quadratic function, Newton's method consists of applying equation 4.12 once to jump to the minimum of the function directly. When f is not truly quadratic but can be locally approximated as a positive definite quadratic, Newton's method consists of applying equation 4.12 multiple times. Iteratively updating the approximation and jumping to the minimum of the approximation can reach the critical point much faster than gradient descent would. This is a useful property near a local minimum, but it can be a harmful property near a saddle point. As discussed in section 8.2.3, Newton's method is only appropriate when the nearby critical point is a minimum (all the eigenvalues of the Hessian are positive), whereas gradient descent is not attracted to saddle points unless the gradient points toward them.

Optimization algorithms that use only the gradient, such as gradient descent, are called **first-order optimization algorithms**. Optimization algorithms that also use the Hessian matrix, such as Newton's method, are called **second-order optimization algorithms** (Nocedal and Wright, 2006).

The optimization algorithms employed in most contexts in this book are applicable to a wide variety of functions, but come with almost no guarantees. Deep learning algorithms tend to lack guarantees because the family of functions used in deep learning is quite complicated. In many other fields, the dominant approach to optimization is to design optimization algorithms for a limited family of functions.

In the context of deep learning, we sometimes gain some guarantees by restricting ourselves to functions that are either **Lipschitz continuous** or have Lipschitz continuous derivatives. A Lipschitz continuous function is a function f whose rate of change is bounded by a **Lipschitz constant** $\mathcal{L}$:

$$\forall \mathbf{x}, \forall \mathbf{y}, |f(\mathbf{x}) - f(\mathbf{y})| \leq \mathcal{L} \|\mathbf{x} - \mathbf{y}\|_2. \quad (4.13)$$

This property is useful because it allows us to quantify our assumption that a small change in the input made by an algorithm such as gradient descent will have

a small change in the output. Lipschitz continuity is also a fairly weak constraint, and many optimization problems in deep learning can be made Lipschitz continuous with relatively minor modifications.

Perhaps the most successful field of specialized optimization is **convex optimization**. Convex optimization algorithms are able to provide many more guarantees by making stronger restrictions. Convex optimization algorithms are applicable only to convex functions—functions for which the Hessian is positive semidefinite everywhere. Such functions are well-behaved because they lack saddle points and all of their local minima are necessarily global minima. However, most problems in deep learning are difficult to express in terms of convex optimization. Convex optimization is used only as a subroutine of some deep learning algorithms. Ideas from the analysis of convex optimization algorithms can be useful for proving the convergence of deep learning algorithms. However, in general, the importance of convex optimization is greatly diminished in the context of deep learning. For more information about convex optimization, see [Boyd and Vandenberghe \(2004\)](#) or [Rockafellar \(1997\)](#).